

Fundamentos de Python

Fundamentos de Ingenieria de Datos

Objetivos de la sesion

Al finalizar la sesion, el alumno podra:

- Reconocer la sintaxis basica de Python.
- Utilizar variables y tipos de datos.
- Construir expresiones y condiciones.
- Trabajar con listas y diccionarios.
- Utilizar ciclos y funciones.
- Leer informacion desde archivos.
- Comprender el uso de librerias.
- Relacionar Python con tareas de ingenieria de datos.
- Comparar conceptos basicos de Python y C++.

Enfoque

Concepto → ejemplo → ejercicio → aplicacion a datos.

Que es Python

Python es un lenguaje de programacion:

- de alto nivel;
- de proposito general;
- con sintaxis relativamente sencilla;
- con un amplio ecosistema de librerias.

Es utilizado en:

Datos

Analisis y procesamiento

IA

Machine Learning

Software

Automatizacion

Pregunta

Por que Python es tan utilizado en Ingenieria de Datos?

Modelo mental: entrada, proceso y salida

ENTRADA	PROCESAMIENTO	SALIDA
Datos	Reglas / código	Información
CSV	Python	Reporte
JSON	Transformaciones	Métricas
Base de datos	Validaciones	Tabla limpia

Ejemplo:

[10, 20, 30, 40] → promedio → 25

Idea clave

En ingeniería de datos no solo importa programar: importa transformar datos de forma reproducible.

1. Variables

Una variable asocia un nombre con un valor.

```
nombre = "Ana"  
edad = 21  
altura = 1.68  
activo = True
```

Tipos comunes:

Tipo	Ejemplo	Uso
int	21	Enteros
float	1.68	Decimales
str	"Ana"	Texto
bool	True	Logica

Para conocer el tipo:

```
type(edad)
```

Ejercicio: variables

Una estacion registra:

```
temperatura = 24.5  
humedad = 62  
ciudad = "Guadalajara"  
lluvia = False
```

Responde:

- 1 Cual variable contiene texto?
- 2 Cuales son numericas?
- 3 Cual contiene un valor booleano?

Ejercicio practico

Agrega:

```
presion = 1013.25
```

Despues verifica su tipo.

2. Operadores

Aritmeticos

+	suma
-	resta
*	multiplicacion
/	division
//	division entera
%	modulo
**	potencia

Comparacion

==	igual
!=	diferente
>	mayor
<	menor
>=	mayor o igual
<=	menor o igual

```
precio = 1000
descuento = 0.15

final = precio * (1 - descuento)

print(final)
```

Ejercicio: operadores

Un producto cuesta \$850 y tiene un descuento del 12 %.
Calcula el precio final.

```
precio = 850
descuento = 0.12

# Completa la formula
final = ...

print(final)
```

Ejercicio practico

Predice primero:

```
x = 17
y = 5

print(x / y)
print(x // y)
print(x % y)
```

3. Condicionales

Los condicionales permiten tomar decisiones.

```
edad = 20

if edad >= 18:
    print("Mayor de edad")
else:
    print("Menor de edad")
```

La estructura es:

if → condicion verdadera

elif → otra condicion

else → ninguna condicion anterior

Importante

La indentacion define el bloque de codigo.

Ejercicio: clasificar temperatura

Clasifica una temperatura:

Valor	Clasificacion
Menor que 10	Fria
10 a 25	Agradable
Mayor que 25	Caliente

Ejercicio practico

Completa:

```
temperatura = 25

if temperatura < 10:
    print("Fria")
elif ...
    ...
else:
    ...
```

4. Listas

Una lista permite almacenar varios elementos.

```
ventas = [120, 150, 90, 210, 180]
```

Python comienza los índices en cero:

Indice	0	1	2	3	4
Valor	120	150	90	210	180

Ejemplos:

```
ventas[0]  
ventas[3]  
ventas[-1]
```

Operaciones con listas

Funciones y metodos utiles:

```
ventas = [120, 150, 90, 210]
```

```
print(len(ventas))
```

```
print(sum(ventas))
```

```
print(min(ventas))
```

```
print(max(ventas))
```

```
ventas.append(300)
```

Operacion	Funcion
Cantidad	<code>len()</code>
Suma	<code>sum()</code>
Minimo	<code>min()</code>
Maximo	<code>max()</code>
Agregar	<code>append()</code>

Ejercicio: analizar ventas

Dada:

```
ventas = [120, 150, 90, 210, 180]
```

Obtiene:

- 1 La primera venta.
- 2 La ultima venta.
- 3 El total.
- 4 La venta maxima.
- 5 La cantidad de registros.

Ejercicio practico

Agrega una nueva venta de 250 y vuelve a calcular el total.

Listas: slicing

Podemos obtener partes de una lista.

```
datos = [10, 20, 30, 40, 50, 60]
```

```
print(datos[1:4])  
print(datos[:3])  
print(datos[3:])  
print(datos[::2])
```

La estructura es:

lista[inicio:fin:paso]

El limite final no se incluye.

Ejercicio practico

Para:

```
temperaturas = [18,20,21,25,27,30,29]
```

obtiene los primeros tres y los ultimos tres elementos.

5. Ciclo for

Un ciclo permite repetir una operacion.

```
ventas = [120, 150, 90, 210]

for venta in ventas:
    print(venta)
```

Mentalmente:

para cada elemento → ejecuta una accion

Ingenieria de datos

Los ciclos permiten procesar registros uno por uno.

Ejercicio: filtrar ventas

Dada:

```
ventas = [120, 150, 90, 210, 180]
```

Muestra solamente ventas mayores que 150.

```
for venta in ventas:  
    if ...:  
        print(...)
```

Ejercicio practico

Modifica la salida para obtener:

```
Venta alta: 210  
Venta alta: 180
```

range()

`range()` genera secuencias de numeros.

```
for i in range(5):  
    print(i)
```

Resultado:

0, 1, 2, 3, 4

Otro ejemplo:

```
for i in range(2, 10, 2):  
    print(i)
```

Resultado:

2, 4, 6, 8

Ejercicio practico

Genera:

6. Funciones

Una funcion agrupa una operacion reutilizable.

```
def calcular_iva(precio):  
    return precio * 0.16  
  
iva = calcular_iva(1000)  
  
print(iva)
```

Una funcion puede tener:

Entrada → procesamiento → salida

Ventaja

Evita repetir la misma logica en diferentes partes del programa.

Ejercicio: funcion promedio

Crea:

```
calcular_promedio(lista)
```

Debe recibir:

```
[8, 9, 10, 7]
```

y regresar:

8.5

Pista:

```
def calcular_promedio(lista):  
    return ...
```

Ejercicio practico

Utiliza `sum()` y `len()`.

7. Diccionarios

Un diccionario almacena pares:

clave → valor

Ejemplo:

```
alumno = {  
    "nombre": "Laura",  
    "edad": 21,  
    "promedio": 9.2  
}
```

Acceso:

```
print(alumno["nombre"])  
print(alumno["promedio"])
```

Comparacion

Lista: acceso principalmente por posicion.

Diccionario: acceso mediante una clave.

Ejercicio: representar un producto

Crea un diccionario para un producto.

Debe tener:

- id
- nombre
- precio
- stock

Ejemplo:

```
producto = {  
    "id": 101,  
    "nombre": "Laptop",  
    "precio": 18000,  
    "stock": 5  
}
```

Ejercicio practico

Imprime:

```
El producto cuesta 18000
```

Datos reales: lista de diccionarios

Una estructura muy frecuente es:

```
ventas = [  
    {"producto": "Laptop", "precio": 18000},  
    {"producto": "Mouse", "precio": 500},  
    {"producto": "Teclado", "precio": 1200}  
]
```

Podemos recorrerla:

```
for venta in ventas:  
    print(venta["producto"])
```

Modelo mental

Un diccionario puede representar un registro.

Una lista puede representar muchos registros.

Ejercicio: registros de ventas

Agrega cantidad:

```
{  
    "producto": "Laptop",  
    "precio": 18000,  
    "cantidad": 2  
}
```

Calcula:

$$\text{importe} = \text{precio} \times \text{cantidad}$$

Ejercicio practico

Recorre todos los registros y muestra:

```
Laptop -> 36000
```

para cada producto.

8. Comprensiones de listas

Forma tradicional:

```
cuadrados = []  
  
for x in range(1, 6):  
    cuadrados.append(x ** 2)
```

Forma compacta:

```
cuadrados = [x ** 2 for x in range(1, 6)]
```

Resultado:

[1, 4, 9, 16, 25]

Ejercicio practico

Crea una lista con los numeros pares del 1 al 10.

```
pares = [...]
```

9. Leer archivos

Los datos pueden provenir de archivos.

```
with open("ventas.txt", "r") as archivo:  
    datos = archivo.readlines()  
  
for linea in datos:  
    print(linea.strip())
```

El patron:

abrir → leer → procesar → cerrar

se utiliza frecuentemente en pipelines.

Buena practica

`with open(...)` gestiona automaticamente el cierre del archivo.

Ejercicio: archivo de alumnos

Crea un archivo llamado:

`alumnos.txt`

con varios nombres.

Lee el archivo:

```
with open("alumnos.txt", "r") as archivo:  
    datos = archivo.readlines()
```

Ejercicio practico

Muestra:

```
Alumno: Ana  
Alumno: Carlos  
Alumno: Laura
```

usando un ciclo.

10. Manejo de errores

Los datos reales pueden contener valores inesperados.

```
try:
    edad = int(input("Edad: "))
    print(edad)

except ValueError:
    print("La edad no es valida")
```

Idea

El programa puede detectar una situación excepcional y responder de manera controlada.

Ejemplos:

- texto donde se esperaba un numero;
- archivo inexistente;
- dato faltante;
- formato incorrecto.

Ejercicio: entrada invalida

El siguiente programa puede fallar:

```
edad = int(input("Edad: "))
```

Que ocurre si el usuario escribe:

hola

Ejercicio practico

Modifica el programa usando:

```
try:  
    ...  
except ValueError:  
    ...
```

para mostrar un mensaje comprensible.

11. Modulos y librerias

Python cuenta con una biblioteca estandar.

Ejemplo:

```
import math

radio = 5

area = math.pi * radio ** 2

print(area)
```

Algunos modulos:

Modulo	Uso
math	Matematicas
statistics	Estadistica
datetime	Fechas
json	JSON
csv	CSV

Ejercicio: estadística

Utiliza:

```
import statistics
```

para calcular el promedio:

```
datos = [8.5, 9.0, 7.5, 10.0, 8.0]
```

Pista:

```
statistics.mean(datos)
```

Ejercicio práctico

Que ventaja tiene utilizar una librería en lugar de implementar cada algoritmo desde cero?

12. Pandas

Para trabajar con datos tabulares es comun utilizar Pandas.

```
import pandas as pd

df = pd.read_csv("ventas.csv")

print(df.head())
```

Conceptualmente:

	producto	precio	cantidad
0	Laptop	18000	2
1	Mouse	500	5
2	Teclado	1200	3

Conceptos

DataFrame = estructura tabular.

Serie = una columna.

Pandas: filtrar informacion

Supongamos:

```
df[df["precio"] > 1000]
```

La condicion selecciona ciertos registros.

Tambien podemos utilizar:

```
df["precio"].max()  
df["precio"].mean()  
df["precio"].sum()  
df.shape
```

Pensamiento de datos

Seleccionar, transformar y resumir son operaciones fundamentales en Ingenieria de Datos.

Ejercicio: analizar una tabla

Supongamos que tenemos:

```
df["precio"]  
df["cantidad"]
```

Realiza:

- 1 Filtrar precios mayores a 1000.
- 2 Obtener el precio maximo.
- 3 Obtener el promedio.
- 4 Obtener la cantidad de registros.

Ejercicio practico

Pregunta:

Que ventajas puede tener Pandas frente a recorrer manualmente miles de registros?

13. Pensamiento de pipeline

Una tarea de Ingenieria de Datos puede organizarse como:



Ejemplo:

CSV → Python → limpieza → metricas → CSV

Pregunta

En que etapa se utilizarian las funciones que aprendimos?

Ejercicio: diseñar un pipeline

Tenemos:

`ventas.csv`

con:

`producto, precio, cantidad, fecha`

Diseña las etapas para:

- 1 Leer.
- 2 Validar.
- 3 Limpiar.
- 4 Calcular importes.
- 5 Obtener metricas.
- 6 Guardar resultados.

Ejercicio practico

Primero diseña el proceso.

Despues escribe el codigo.

Ejercicio integrador: Mini Pipeline de Ventas

Utiliza los siguientes datos:

```
ventas = [  
    {"producto": "Laptop",  
     "precio": 18000,  
     "cantidad": 2},  
  
    {"producto": "Mouse",  
     "precio": 500,  
     "cantidad": 5},  
  
    {"producto": "Teclado",  
     "precio": 1200,  
     "cantidad": 3},  
  
    {"producto": "Monitor",  
     "precio": 4500,  
     "cantidad": 2},  
  
    {"producto": "USB",  
     "precio": 250,  
     "cantidad": 8}
```

Integrador: requisitos

El programa debe:

- 1 Recorrer las ventas.
- 2 Calcular el importe de cada registro.
- 3 Calcular el total vendido.
- 4 Encontrar el mayor importe.
- 5 Calcular el promedio de precios.
- 6 Mostrar productos con importe mayor a 2000.
- 7 Utilizar al menos una funcion.

Formula:

$$\text{importe} = \text{precio} \times \text{cantidad}$$

Reto

Crea una funcion que reciba la lista y produzca un resumen de las ventas.

Integrador: plantilla inicial

Puedes comenzar con:

```
def analizar_ventas(ventas):  
  
    total = 0  
    mayor = 0  
    promedio = 0  
  
    for venta in ventas:  
  
        importe = (  
            venta["precio"]  
            * venta["cantidad"]  
        )  
  
        # Completa el procesamiento  
  
    return total, mayor, promedio
```

Ejercicio practico

Completa la funcion sin consultar la solucion.

Integrador: resultados esperados

Los importes son:

Producto	Precio	Cantidad	Importe
Laptop	18000	2	36000
Mouse	500	5	2500
Teclado	1200	3	3600
Monitor	4500	2	9000
USB	250	8	2000

Total = 53100

Mayor importe = 36000

Promedio de precios = 4890

Integrador: llevarlo a datos reales

Ahora analiza:

- 1 Que ocurre si falta un precio?
- 2 Que ocurre si la cantidad es negativa?
- 3 Que ocurre si existen 100000 registros?
- 4 Como leerias un CSV?
- 5 Como guardarias los resultados?
- 6 Que parte podria realizar Pandas?

Cambio de perspectiva

Ya no se trata solamente de programar.

Se trata de construir un proceso reproducible sobre datos.

Python vs C++: variables

Python

```
edad = 21  
nombre = "Ana"  
altura = 1.68
```

El tipo se determina durante la ejecución.

C++

```
int edad = 21;  
string nombre = "Ana";  
double altura = 1.68;
```

El tipo se declara explícitamente.

Concepto	Python	C++
Entero	int	int
Decimal	float	double
Texto	str	string
Booleano	bool	bool

Python vs C++: listas

Python

```
ventas = [100, 200, 300]

ventas.append(400)

for x in ventas:
    print(x)
```

C++

```
vector<int> ventas =
    {100, 200, 300};

ventas.push_back(400);

for (int x : ventas) {
    cout << x;
}
```

Operacion	Python	C++
Coleccion	list	vector
Agregar	append()	push_back()
Tamano	len()	size()

Python vs C++: condicionales

Python

```
if edad >= 18:  
    print("Adulto")  
else:  
    print("Menor")
```

C++

```
if (edad >= 18) {  
    cout << "Adulto";  
}  
else {  
    cout << "Menor";  
}
```

Diferencia

Python utiliza indentación.

C++ utiliza principalmente llaves.

Python vs C++: funciones

Python

```
def promedio(datos):  
    return sum(datos) / len(datos)
```

C++

```
double promedio(  
    vector<double> datos) {  
  
    double suma = 0;  
  
    for (double x : datos)  
        suma += x;  
  
    return suma / datos.size();  
}
```

Observacion

La idea es la misma: encapsular una operacion reutilizable.

Python vs C++: diccionarios

Python

```
alumno = {  
    "nombre": "Laura",  
    "edad": 21  
}  
  
print(alumno["nombre"])
```

C++

```
map<string, string> alumno;  
  
alumno["nombre"] = "Laura";  
  
cout << alumno["nombre"];
```

Nota

C++ dispone de estructuras como `map` y `unordered_map`; no son idénticas al diccionario de Python.

Python vs C++: librerías

Necesidad	Python	C++
Matematicas	math	cmath
Estadística	statistics	Bibliotecas externas o código propio
CSV	csv / pandas	fstream o bibliotecas externas
JSON	json	Bibliotecas externas
Datos tabulares	pandas	Sin equivalente directo en biblioteca estándar
Cálculo numérico	NumPy	Eigen, BLAS u otras
Visualización	Matplotlib	Bibliotecas externas

Idea

El ecosistema de Python es una de las razones de su presencia en ciencia de datos e Ingeniería de Datos.

Comparativa rapida Python vs C++

Concepto	Python	C++
Variable	<code>x = 10</code>	<code>int x = 10;</code>
Lista	<code>list</code>	<code>vector</code>
Diccionario	<code>dict</code>	<code>map</code>
Ciclo	<code>for x in datos:</code>	<code>for (auto x : datos)</code>
Condicion	<code>if condicion:</code>	<code>if (condicion)</code>
Funcion	<code>def funcion():</code>	<code>tipo funcion()</code>
Archivo	<code>open()</code>	<code>fstream</code>
Datos tabulares	<code>pandas.DataFrame</code>	No existe equivalente estandar directo
Memoria	Gestion automatica	Mayor control explicito
Ejecucion	Runtime de Python	Codigo compilado

Que aprendimos

Python

- Variables
- Tipos
- Operadores
- Condiciones
- Listas
- Diccionarios

Procesamiento

- Ciclos
- Funciones
- Archivos
- Excepciones
- Librerías
- Pandas

Python → Procesamiento → Datos

Preguntas de cierre

- 1 Que diferencia existe entre una lista y un diccionario?
- 2 Cuando conviene crear una funcion?
- 3 Que problema resuelve un ciclo?
- 4 Como representar un registro?
- 5 Que ventaja ofrece Pandas?
- 6 Que diferencias existen entre Python y C++?
- 7 Como convertirias el integrador en un pipeline real?

Pregunta final

Si mañana recibieras un archivo con un millon de registros, que parte del proceso automatizarias primero?

Fundamentos de Python

Ingenieria de Datos

Datos → Código → Transformación → Información

El objetivo no es memorizar Python.
Es aprender a resolver problemas con datos.